AI Scanner

AI-based document scanner: upload a document and get its type, key metadata and expiry status.

1. Overview

AI Scanner has two parts: a Python **FastAPI backend** that reads and understands documents (OCR, classification, metadata and expiry extraction), and a **Laravel frontend** where users upload files and see the result.

```
AiScanner/
  backend/   FastAPI document parser
    app/         API (main.py), Redis/RQ queue (job_queue.py, tasks.py), config
    parser/      file loading, OCR, text cleaning, parsing service
    classification/  keyword + embedding (hybrid) classifier, country detector
    metadata/    company, people, vehicle, equipment extractors
    expiry/      date extraction and expiry status
    data/        category keywords, training/evaluation samples
    samples/, docs/, scripts/, tests/
  frontend/  Laravel web UI
```

2. How a document is processed

• **Load** - file is validated (max 10 MB) and text extracted. PDFs and images use Tesseract OCR (200 DPI, first 5 pages); Word/PowerPoint/TXT files are read directly.

• **Clean** - extracted text is normalised.

• **Classify** - hybrid classifier: keyword matching first, then sentence-embedding similarity (all-MiniLM-L6-v2), then filename hints as a fallback. The result reports the method used.

• **Country** - detects India, UK, US or EU from the content.

• **Metadata** - routes to extractors for company, people, vehicle or equipment details depending on document type.

• **Expiry** - finds start and expiry dates and reports status: valid, expiring_soon, expired or unknown, with days remaining.

3. Supported files

PDF, JPG/JPEG, PNG, TIF/TIFF, DOCX/DOC, PPTX/PPT, TXT. Maximum size 10 MB.

4. API endpoints (backend, port 8000)

Method	Path	Purpose
GET	/health	Health check
GET	/	Service info and endpoint list
POST	/parse-document	Upload a file; returns the result immediately
POST	/jobs	Upload a file; queued in the background, returns job_id (HTTP 202)
GET	/jobs/{job_id}	Job status: queued, started, finished (with result) or failed

POST	/debug-keywords	Per-category keyword match scores, for diagnostics

Interactive API docs are available at **/docs** while the backend runs.

Response of /parse-document

```
{
  "success": true,
  "document_type": "...",
  "confidence": 0.0 - 1.0,
  "keywords_detected": [...],
  "text_length": 1234,
  "processing_time_ms": 850,
  "method": "keyword | embedding | filename | none",
  "country": "India | UK | US | EU | null",
  "metadata": { ... },
  "expiry": { "start_date", "expiry_date", "days_remaining", "status" }
}
```

Errors return `{success:false, error_code, message}`: unsupported_file_type / invalid_file (400), corrupt_or_unreadable_file / ocr_failure (422), queue_unavailable (503), job_not_found (404), internal_error (500).

5. Background job queue

Large volumes are handled with Redis and RQ. The API enqueues a document, workers on the *parse* queue process it, and the client polls `GET /jobs/{id}`. Running more workers processes more documents in parallel. Redis URL: `REDIS_URL` (default redis://127.0.0.1:6379/0).

6. Running the project

Terminal 1 - backend (port 8000)

```
cd backend
source venv/bin/activate
uvicorn app.main:app --port 8000
```

Terminal 2 - worker (Redis must be running: brew services start redis)

```
cd backend
source venv/bin/activate
rq worker parse --worker-class rq.SimpleWorker
```

Terminal 3 - frontend (port 8001)

```
cd frontend
./serve.sh 8001
```

Open http://127.0.0.1:8001. The backend URL is set by `PARSER_URL` in frontend/.env.

7. Tech stack

Area	Technology
Backend API	Python, FastAPI, Uvicorn
OCR / files	Tesseract (pytesseract), pdf2image, Pillow, pdfplumber, python-docx, python-pptx
Classification	sentence-transformers, scikit-learn

Queue	Redis, RQ
Frontend	Laravel (PHP)
Tests	pytest